五、完整RAG搭建

在前面的课程中，我们已经实现过一个基于 Chroma 的简单 RAG 系统。那时我们主要关注如何把文档存入向量数据库并进行检索，代码相对比较直接。

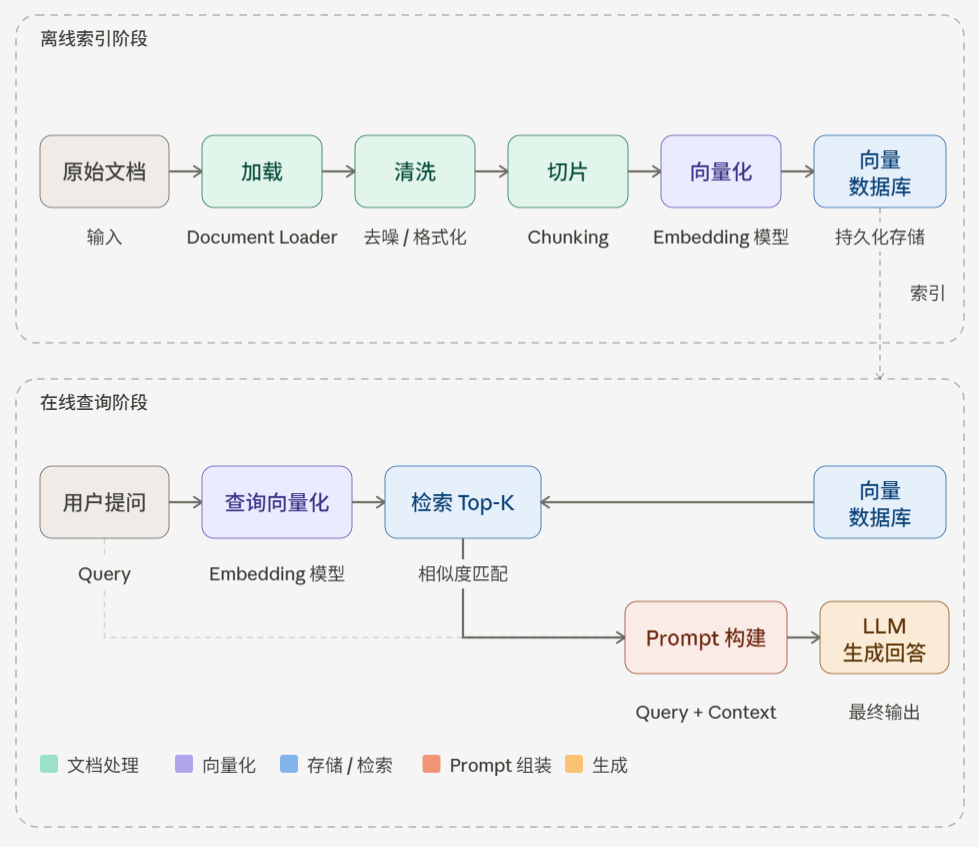
然而，那个版本存在明显的不足：

- 缺少对原始数据的**清洗和预处理**
- 没有进行合理的**文本切片（Chunking）**
- 没有清晰地区分**离线索引**和**在线查询**两个阶段
- 整体代码结构较为混乱，难以维护和扩展

而一个真正可用的 RAG 系统，通常需要包含两个核心阶段：

- **离线索引阶段**：对原始文档进行清洗、切分、向量化，并存入向量数据库。
- **在线查询阶段**：接收用户问题，从向量数据库中检索相关内容，再结合 Prompt 让大模型生成回答。

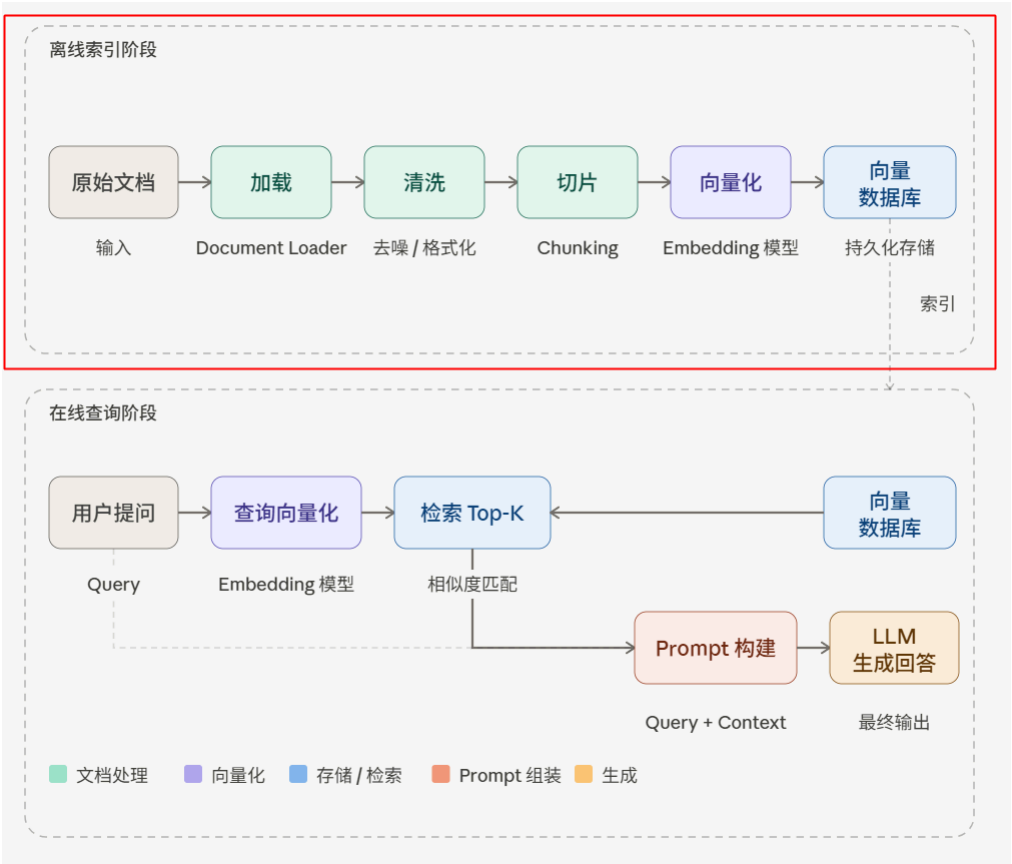
本章我们将基于 **LCEL**，重新构建一个结构清晰、流程完整的**朴素RAG系统**。通过本次实现，我们将系统地掌握 RAG 的完整构建流程，并为后续学习更高级的 RAG 技术打下基础。



1.离线索引阶段

1.1 概述

在构建 RAG 系统时，我们通常会把整个流程分为两个主要阶段：离线索引阶段 和 在线查询阶段。本章我们先聚焦于离线索引阶段。



1.1.1 什么是离线索引

离线索引阶段（Offline Indexing）是指在系统正式对外提供服务之前，对原始数据进行处理，并将其转化为可被高效检索的形式的过程。

简单来说，离线索引就是"**把知识提前准备好**"的阶段。这个阶段通常在后台完成，不需要实时响应用户请求，因此我们可以投入更多计算资源进行数据清洗、切片、向量化等相对耗时的操作。

离线索引阶段的核心目标是：

- 将非结构化或半结构化的原始文档，转化为结构化的、可向量化的数据
- 构建高质量的向量数据库，为后续的在线检索提供支持
- 尽可能在这一阶段完成数据清洗和优化，减少在线查询时的负担

1.1.2 流程概述

离线索引阶段一般包含以下几个核心步骤：

文档加载（Document Loading）

- 从不同来源（如 PDF、Word、网页、数据库等）加载原始文档

数据清洗（Data Cleaning）

- 去除噪声、无用信息、格式错误等
- 统一文本格式、处理特殊字符、去除重复内容等

文本切片（Text Chunking）

- 将长文档切分成适合向量化的较小片段（Chunk）
- 这是非常关键的一步，直接影响后续检索效果

向量化（Embedding）

- 使用 Embedding 模型将文本片段转换为向量表示

存储到向量数据库（Vector Store）

- 将向量及元数据存入 Chroma、FAISS、Milvus 等向量数据库中
- 通常还会设置自定义 ID，便于后续管理和更新

离线索引阶段的整体流程图：

```
原始文档 → 加载 → 清洗 → 切片 → 向量化 → 存入向量数据库
```

这个阶段完成后，我们就拥有了一个可供检索的知识库。接下来在**在线查询阶段**，我们就可以基于这个知识库快速响应用户问题。

1.2 文档加载

文档加载是 RAG 系统的第一步。加载质量直接影响后续切片和检索的效果。在实际项目中，我们经常需要处理多种格式的文档，其中 PDF 是最常见但也最容易出问题的格式。

1.2.1 加载方式

在之前的课程中，我们已经学习了 LangChain 提供的多种 **Document Loader**，用于加载不同格式的文档。这些加载器将不同数据源的读取逻辑进行了封装，让我们可以相对统一的方式获取 Document 对象。

（1）常见的加载器：

- TextLoader：用于加载 TXT 和 Markdown 文件
- PyPDFLoader：用于加载 PDF 文件
- WebBaseLoader：用于加载网页内容
- JSONLoader：用于加载 JSON 格式的数据

这些加载器大大简化了文档加载的流程，我们可以通过相对一致的接口，将不同来源的文档转换为 LangChain 统一的 Document 格式，方便后续进行切片和向量化。

（2）复杂场景

然而，在实际项目中，我们经常会遇到**排版复杂的 PDF 文档**，例如双栏排版的法律条文、学术论文、技术报告等。此时使用 PyPDFLoader 等常规加载器，往往无法获得理想的提取效果。

这主要是因为复杂 PDF 在排版上存在多栏、表格、特殊布局等问题，普通加载器难以正确识别文字的阅读顺序和结构。这也是我们接下来需要重点关注的问题。

（3）**双栏排版对文本解析带来的影响**

虽然双栏排版在印刷和阅读上有其优势，但对于我们的 RAG 系统数据处理却带来了显著挑战：

- 通用 PDF 解析工具（如 PyPDF、PyMuPDF、Unstructured 等）在读取时容易出现**段落断裂、阅读顺序错误、文本混乱**等问题；
- 跨栏内容经常被错误拼接，导致语义不连贯；
- 后续的文本清洗和切片效果大幅下降，最终影响向量化质量和检索准确率。

1.2.2 常用 PDF 加载器对比

在 LangChain 生态中，处理 PDF 文档时有多种加载器可供选择。不同的加载器底层实现不同，在处理**双栏排版、表格、复杂布局**时的表现差异较大。下面我们对几种常用的 PDF 加载器进行对比：

1. `PyPDFLoader`

- **所属库**：`langchain_community.document_loaders`
- **底层依赖**：`pypdf`
- **优点**：
    - 使用简单，LangChain 原生支持
    - 加载速度较快
- **缺点**：
    - 对双栏、多栏排版支持较差，容易出现文本顺序混乱
    - 表格识别能力弱
    - 元数据提取相对有限
- **适用场景**：简单排版的单栏 PDF

2. `PyMuPDFLoader`

- **所属库**：`langchain_community.document_loaders`
- **底层依赖**：`PyMuPDF`（`fitz`）
- **优点**：
    - 对复杂排版（包括双栏）的支持明显优于 `PyPDFLoader`
    - 可以较好地保留页面结构
    - 支持提取图片和表格（需额外配置）
- **缺点**：
    - 需要额外安装 `pymupdf`
    - 输出仍然是纯文本，结构化能力一般
- **适用场景**：中等复杂度的 PDF 文档

3. `PyMuPDF4LLMLoader`（推荐）

- **所属库**：`langchain_pymupdf4llm`
- **底层依赖**：`pymupdf4llm`
- **优点**：
    - 专为 LLM/RAG 场景优化，对双栏、表格、复杂排版的处理效果较好
    - 默认输出更适合大模型理解的 Markdown 格式
    - 与 LangChain Loader 接口完全兼容，使用方便
- **缺点**：
    - 相对较新，社区案例和文档仍在完善中
    - 处理极度复杂的版面时仍可能需要额外调优
- **适用场景**：**推荐用于朴素 RAG 项目**，尤其是法律、学术、合同类文档

4. `UnstructuredPDFLoader`

- **所属库**：`langchain_community.document_loaders`
- **底层依赖**：`unstructured`
- **优点**：
    - 对复杂文档的解析能力较强（支持多种元素识别）
    - 可以提取表格、图片标题等结构化信息
- **缺点**：
    - 安装依赖较多，部署成本较高
    - 加载速度相对较慢
    - 对双栏 PDF 的处理效果因版本而异
- **适用场景**：需要提取结构化信息的场景（进阶使用）

总结对比表

加载器	双栏 PDF 支持	输出格式	易用性	推荐指数（朴素 RAG）	备注
`PyPDFLoader`	差	纯文本	高	★☆☆☆☆	简单文档可用
`PyMuPDFLoader`	中	纯文本	高	★★★☆☆	比 PyPDF 好一些
`PyMuPDF4LLMLoader`	好	Markdown	高	★★★★★	**推荐**
`UnstructuredPDFLoader`	中上	结构化元素	中	★★★☆☆	较重

1.2.3 解决方案

针对双栏 PDF 解析效果差的问题，我们推荐使用 `PyMuPDF4LLMLoader` 作为解决方案。它是 LangChain 官方维护的 Partner Package，底层基于 `pymupdf4llm` 开发，专门针对大模型（LLM）和 RAG 场景进行了优化。

1. 主要特点

- **对复杂排版支持更好**：相比传统的 `PyPDFLoader`，它在处理双栏、多栏、表格等复杂布局时，文本提取质量明显更高。
- **输出更适合 LLM**：默认会尽量输出结构化程度更高的内容（通常为 Markdown 格式），更有利于后续的文本切片和向量化。
- **与 LangChain 生态无缝衔接**：返回标准的 `Document` 对象，可以直接用于后续的切片、嵌入和检索流程。
- **使用简单**：API 设计与 LangChain 其他 Loader 保持一致，上手成本低。

2. 工作原理（简要说明）

`PyMuPDF4LLMLoader` 底层使用 **PyMuPDF** 强大的 PDF 解析引擎，结合 `pymupdf4llm` 针对 LLM 场景做的额外优化处理：

- 它会尝试识别文档的布局结构（而非简单按顺序提取文字）
- 对双栏内容进行更合理的排序和合并
- 将提取结果整理成更适合大模型理解的 Markdown 格式
- 保留页码、来源等元数据信息

因此，它在复杂 PDF 上的表现通常优于普通的 `PyPDFLoader` 和 `PyMuPDFLoader`。

3. 加载后的输出形式

使用 `PyMuPDF4LLMLoader` 加载后，会返回 `List[Document]`，每个 `Document` 包含：

- `page_content`：提取后的文本内容（通常为 Markdown 格式）
- `metadata`：包含 `source`（文件路径）、`page`（页码）等信息

这种输出格式非常便于后续进行文本切片和构建向量数据库。

4. 使用示例

```python
from langchain_pymupdf4llm import PyMuPDF4LLMLoader

# 初始化加载器
loader = PyMuPDF4LLMLoader(
    file_path="./法律条文.pdf",
    # 可选参数
    pages=None,           # None 表示加载全部页面
    extract_images=False  # 是否提取图片（默认 False）
)

# 执行加载
docs = loader.load()

print(f"共加载 {len(docs)} 个 Document")
```

```python
print("第一页内容预览：\n")
print(docs[0].page_content[:600])
print("\n元数据示例：")
print(docs[0].metadata)
```

5. 适用场景建议

场景	是否推荐使用 `PyMuPDF4LLMLoader`	说明
普通单栏 PDF	可选	普通加载器也能满足
双栏/复杂排版 PDF	**强烈推荐**	效果明显优于其他 Loader
需要 Markdown 输出	推荐	默认输出更结构化
对加载速度要求极高	不推荐	相比原生 PyMuPDF 稍慢一些

1.3 数据清洗

1.3.1 概述

（1）什么是数据清洗

数据清洗是指对原始文档进行处理，去除其中无用、混乱或干扰信息的过程，使文本变得更干净、结构更清晰。

在 RAG 系统中，原始数据（尤其是 PDF、网页等）通常包含很多噪声，比如多余的空格、换行、页码、页眉页脚、特殊符号、乱码等。数据清洗的目的就是把这些"脏数据"处理掉，让文档更适合后续的切片、向量化以及模型使用。

简单来说，**数据清洗就是把原始文档"洗干净"**，为后面的流程打好基础。

（2）数据清洗的作用

数据清洗在 RAG 中起到的作用主要有以下几点：

- **提高向量化质量**：干净的文本能让 Embedding 模型更好地理解语义，生成的向量质量更高。
- **提升检索准确率**：去除无关信息后，系统能更精准地检索到与用户问题相关的片段。
- **改善模型生成效果**：模型在回答问题时，如果参考的内容本身比较干净、有条理，生成的答案也会更准确、更有逻辑。
- **降低后续处理难度**：清洗后的文本在进行切片时，更容易切出语义完整的段落，也方便后续提取元数据。

总结：

数据清洗虽然是 RAG 最前面的步骤，但它对整个系统的最终效果影响很大。如果这一步做得不好，后面的检索和生成再努力，效果也很难达到理想水平。

1.3.2 第一层：基础清洗

1.去除多余空格

PDF或网页解析后，经常会出现多个连续空格的情况。这些多余的空格虽然不影响阅读，但会让文本显得不整洁，也可能影响后续的文本切片效果。使用正则表达式可以将多个空格合并为一个空格。

```python
import re

# 示例文本
text = "AI      is coming"

# 替换多个空格为一个空格
text = re.sub(r"\s+", " ", text)

print(text)
# 输出: "AI is coming"
```

2.去除多余换行

很多文档在解析后会出现大量连续的空行，这些空行没有实际意义。保留必要的段落换行（一个换行），删除多余的空行，可以让文本结构更清晰，也能减少无效内容进入向量数据库。

```python
# 示例文本
text = "人工智能\n\n是未来的发展方向\n\n\n机器学习是其重要分支"

# 连续换行替换成一个换行
text = re.sub(r"\n+", "\n", text)

print(text)
# 输出:
# 人工智能
# 是未来的发展方向
# 机器学习是其重要分支
```

> 保留必要的段落换行，删除重复空行。

3.去除特殊字符

文档中经常存在一些没有实际意义的特殊字符，比如 @、#、*、以及 OCR 识别产生的乱码符号（如 ■、□）。这些字符不仅没有信息价值，还可能干扰向量化模型的理解，因此需要清理掉。

```python
# 示例文本
text = "@@@@@\n*****\n人工智能是未来的方向\n#####"

# 删除常见特殊字符
text = re.sub(r"[@#*■□]+", "", text)

print(text)
# 输出:
# 人工智能是未来的方向
```

> 可以根据实际文档扩展正则字符集。

1.3.3 捕获组

捕获组（Capturing Groups）是正则表达式中用于**标记子模式**并**记录匹配内容**的核心机制。通过捕获组，可以将正则模式中的某一部分提取出来，并在后续匹配或替换过程中重复使用。

（1）基本语法

- **捕获组**：使用一对圆括号 () 包裹需要捕获的子模式。
- **编号规则**：从左至右按出现顺序自动编号，从 1 开始（第 0 组始终为整个匹配内容）。
- **反向引用**：在同一模式中使用 \1、\2 等引用已捕获的内容；或在 re.sub 的替换字符串中使用 \1、\2。

语法形式：

- (pattern)：捕获组，匹配 pattern 并记录匹配结果。
- \1：引用第 1 个捕获组的内容。
- \2：引用第 2 个捕获组的内容。

（2）实际示例

```python
import re

text = "小坤今年18岁，他喜欢唱跳RAP打篮球。"

# 示例1：捕获并提取数字和单位
result = re.search(r'(\d+)岁', text)
print(result.group(0))   # 整个匹配：18岁
print(result.group(1))   # 第1组：18

# 示例2：同时捕获两个部分
result2 = re.search(r'(\w+)喜欢(\w+)', text)
print(result2.group(1))  # 他
print(result2.group(2))  # 唱跳RAP打篮球
```

（3）反向引用在替换中的应用（最常见场景）

```python
text = "atten- tion 以及  pro- cess"

# 使用捕获组 + 反向引用修复断裂单词
cleaned = re.sub(r'(\w+)-\s*(\w+)', r'\1\2', text)
print(cleaned)   # 输出：attention 以及 process
```

解释：

- (\w+)：第1组，匹配前半部分单词。
- -\s*：匹配连字符和空白。
- (\w+)：第2组，匹配后半部分单词。
- 替换字符串 r'\1\2'：将第1组和第2组的内容直接拼接，消除断裂。

1.3.4 第二层：标准清洗

1. 去除页码

很多 PDF 文档每一页都会带有页码（如“第1页”、“Page 1”等）。如果不删除这些页码，切片后向量库中会出现大量重复且无意义的页码信息，影响检索效果。

```python
# 示例文本
text = "第1页\n人工智能是未来的方向\n第2页\n机器学习是其分支"

# 删除格式类似 “第1页”、“第 12 页”、“Page 1”的页码
text = re.sub(r"(第\s*\d+\s*页|Page\s*\d+)", "", text)
# | 在正则表达式中表示 “或”（OR）。
# () 在正则表达式中叫分组（Group），也叫捕获组。

print(text)
# 输出:
# 人工智能是未来的方向
# 机器学习是其分支
```

2. 去除页眉页脚

页眉和页脚通常是每页都重复出现的内容（如公司名称、版权声明、日期等）。这些内容对用户的问题基本没有帮助，属于典型的噪声数据，建议在清洗阶段删除。

> 页眉页脚通常在每页都相同，可以用最简单的办法：匹配固定字符串或开头/结尾内容。

```python
import re

text = """
XX公司内部培训资料
第1页
人工智能是未来方向
版权所有 © 2026
XX公司内部培训资料
```

```
第2页
机器学习是其分支
版权所有 © 2026
"""

# 匹配每页开头的固定页眉 + 页码，结尾的固定页脚
text = re.sub(
    r"(XX公司内部培训资料\s*第\d+页\s*|版权所有 © 2026\s*)",
    "",
    text
)

# 去掉每行多余空格和空行
text = "\n".join([line.strip() for line in text.splitlines() if line.strip() != ""])

print(text)
```

3. 换行切断修复（Line Break / Sentence Merge）

很多 PDF 文档（尤其是双栏排版）会将一个完整的单词拆成两行，中间用连字符“-”连接，例如“atten- tion”。如果不将这些断裂的单词合并，检索时会被拆成无意义的片段，影响召回效果。

```
# 示例文本
text = "This is an atten- tion test.\nAnother exam- ple here."

# 修复断裂的单词：匹配 字母- 换行空格 字母 → 直接拼接
text = re.sub(r'(\w+)-\s*\n\s*(\w+)', r'\1\2', text)

print(text)
# 输出：
# This is an attention test.
# Another example here.
```

4. 哈希去重（Exact Hash Deduplication）

哈希去重是一种**精确去重**方法，核心思想是通过哈希函数（如MD5、SHA-1）将文本内容映射为固定长度的哈希值（指纹）。不同的内容（哪怕只有一个字符不同）产生的哈希值也几乎必然不同；内容完全相同则哈希值一定相同。因此，通过维护一个已见哈希值的集合，即可快速判断新内容是否为重复副本，从而达到去重目的。

在RAG清洗中，可以在文档级或段落级执行：

- 对每个段落的前150个字符计算 MD5 哈希值（特征提取，避免整段计算开销）。
- 若哈希值未出现过，则保留该段落；否则丢弃。
- 最后将去重后的段落用 `\n\n` 重新拼接。

举例说明：

假设原始文本如下：

```python
text = '''
人工智能是未来方向。
机器学习是重要分支。
人工智能是未来方向。
深度学习正在崛起。
'''

import hashlib


def deduplication(text: str) -> str:
    paragraphs = [p.strip() for p in text.split('\n')]
    seen = {}
    cleaned = []

    for para in paragraphs:
        # 特征提取：MD5哈希 + 前150字符
        key = hashlib.md5(para[:150].encode('utf-8')).hexdigest()
        if key not in seen:
            seen[key] = para
            cleaned.append(para)

    return '\n'.join(cleaned)


text = deduplication(text)
print(text)
```

经过哈希去重后，第二个重复的段落“人工智能是未来方向。”会被丢弃，输出结果为：

```
人工智能是未来方向。
机器学习是重要分支。
深度学习正在崛起。
```

1.3.5 汇总

根据前面的多个清洗示例，我将它们整合为两个清洗函数：

- `basic_clean(text)`：第一层基础清洗，包含去除多余空格、去除多余换行、去除特殊字符。
- `standard_clean(text)`：第二层标准清洗，包含去除页码、去除页眉页脚、换行切断修复、哈希去重（段落级精确去重）。

下面是完整代码：

```python
import re
import hashlib

# ========== 第一层：基础清洗 ==========
def basic_clean(text: str) -> str:
    """
```

```
    基础清洗：去除多余空格、多余换行、特殊字符
    """
    # 1. 去除多余空格（多个连续空格合并为一个）
    text = re.sub(r"\s+", " ", text)

    # 2. 去除多余换行（多个连续换行合并为一个）
    text = re.sub(r"\n+", "\n", text)

    # 3. 去除常见特殊字符（@、#、*、■、□ 等，可根据需要扩展）
    text = re.sub(r"[@#*■□]+", "", text)

    return text

# ========== 第二层：标准清洗 ==========
def standard_clean(text: str) -> str:
    """
    标准清洗：去除页码、去除页眉页脚、修复换行切断、哈希去重（段落级）
    """
    # 1. 去除页码（支持“第1页”、“第 12 页”、“Page 1”等格式）
    text = re.sub(r"(第\s*\d+\s*页|Page\s*\d+)", "", text)

    # 2. 去除页眉页脚（示例：去除固定页眉“XX公司内部培训资料”、页脚“版权所有 © 2026”）
    #    实际使用时可根据文档特征调整正则表达式
    text = re.sub(
        r"(XX公司内部培训资料\s*第\d+页\s*|版权所有 © \d{4}\s*)",
        "",
        text
    )
    # 清理因删除页眉页脚产生的多余空行和首尾空格
    lines = [line.strip() for line in text.splitlines() if line.strip() != ""]
    text = "\n".join(lines)

    # 3. 换行切断修复（修复单词被连字符换行切断，如 "atten- tion" -> "attention"）
    text = re.sub(r'(\w+)-\s*\n\s*(\w+)', r'\1\2', text)

    # 4. 哈希去重（段落级精确去重，按双换行分割段落，取前150字符计算MD5）
    #    如果原始文本没有双换行作为段落分隔，可先按单换行分割（此处采用单换行，保持与示例一致）
    paragraphs = [p.strip() for p in text.split('\n') if p.strip()]
    seen = {}
    cleaned_paragraphs = []
    for para in paragraphs:
        # 特征提取：MD5哈希 + 前150字符
        key = hashlib.md5(para[:150].encode('utf-8')).hexdigest()
        if key not in seen:
            seen[key] = para
            cleaned_paragraphs.append(para)

    return '\n'.join(cleaned_paragraphs)


# ========== 使用示例 ==========
if __name__ == "__main__":
    raw_text = """
```

```
    @@@@@
    *****
    XX公司内部培训资料
    第1页
    人工智能        是未来的方向。
    atten- tion 是重要概念。
    人工智能是未来的方向。
    第2页
    机器学习是其分支。
    版权所有 © 2026
    """

    cleaned = basic_clean(raw_text)
    cleaned = standard_clean(cleaned)
    print(cleaned)
```

说明：

- `basic_clean` 负责最基础的文本规范化：空格、换行、特殊符号。
- `standard_clean` 负责更高级的噪声移除：页码、页眉页脚、断裂单词、重复段落。
- 哈希去重部分按**单换行分割**（与您提供的示例一致），实际应用时建议根据文档结构调整为双换行分割。

1.4 文本切片

在工程实践中，文本切片策略的选择与实现通常借助成熟的工具库完成，以降低开发成本并保证稳定性。LangChain作为 RAG 应用开发的主流框架，其提供的`langchain-text-splitters`模块封装了多种文本分割策略，成为工业界的标准实践。

`langchain-text-splitters`是LangChain官方维护的文本分割工具集，保持稳定。该模块的核心设计思想是：将文本分割抽象为“分割器（Splitter）”接口，开发者通过配置参数即可获得不同策略的切片器实例，无需关心底层实现细节。

1.4.1 固定长度切片

（1）策略核心思想

固定长度切片是最基础的切片策略，按照预设的字符数将文本均匀分割。尽管在工业级 RAG 系统中，固定长度切片很少作为唯一策略使用，但其设计思想、参数配置和边界效应是理解所有高级切片策略的基础。

不考虑文本的语义边界，直接按固定长度截断。通常配合重叠窗口（overlap）使用，将相邻切片的尾部/头部部分内容重叠，防止关键信息恰好落在截断边界处。重叠窗口的比例通常取切片长度的10%~20%，过低则边界保护失效，过高则导致存储冗余和检索时的信息重复。

（2）代码实现

```
import numpy as np
# 1.固定长度切片
from langchain_text_splitters import CharacterTextSplitter

# 创建固定长度切片器（基于字符）
splitter = CharacterTextSplitter(
```

```
    chunk_size=500,           # 每个切片的字符数
    chunk_overlap=50,         # 重叠字符数
    separator="",             # 分隔符设为空，按字符均匀切割
    length_function=len,      # 使用字符计数
)

with open('./百度千帆.txt','r',encoding='UTF-8') as f:
    text = f.read()
chunks = splitter.split_text(text)

# 统计每个切片的字符数
chunk_lengths = [len(chunk) for chunk in chunks]

# 基本统计量
print(f"切片数量: {len(chunks)}")
print(f"最小长度: {np.min(chunk_lengths)} 字符")
print(f"最大长度: {np.max(chunk_lengths)} 字符")
print(f"平均长度: {np.mean(chunk_lengths):.1f} 字符")
print(f"第一个切片内容:\n{chunks[0]}")
```

1.4.2 递归切片

递归切片是 LangChain 的 `RecursiveCharacterTextSplitter` 所实现的策略，也是当前工业界 RAG 系统中最广泛采用的默认切片方案。其核心优势在于通过多级分隔符的递归降级机制，在语义完整性与粒度可控性之间取得最优平衡。

（1）策略核心思想

递归切片维护一个"分割符优先级"列表，从最粗粒度的语义边界（如段落）到最细粒度的边界（如词、字符）依次降级。其执行逻辑如下：

步骤一：按当前优先级分割符切分

使用当前层级的分割符（如 `\n\n`）将文本分割为多个块。

步骤二：检查每个块的长度

对于每个块，判断其字符数是否超过预设的 `chunk_size` 阈值：

- 若未超过，保留该块，进入最终输出
- 若超过，使用下一级分割符（如 `\n`）对该块递归执行步骤一

步骤三：递归降级

重复上述过程，直到所有块均满足长度要求，或已使用到最后一个分割符。

步骤四：最终降级

若使用最后一个分割符（如空格或空字符串）后，仍有块超过 `chunk_size`，则按固定长度强制截断。

该机制的数学本质是**分治递归**：每次分割将问题规模减小，递归深度受分割符层级数量限制（通常为5~10层），确保算法收敛。

（2）适用场景

- **混合类型文档、未知结构文本**：作为默认策略，无需预先识别文档类型
- **需要兼顾效率与语义完整性的通用场景**：大多数 RAG 系统的首选方案
- **作为专用处理器的降级兜底**：在文档类型识别失败时，递归切片可保证基本质量

（3）代码实践

```
# 2.循环递归切片
from langchain_text_splitters import RecursiveCharacterTextSplitter
import numpy as np


splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,                         # 目标切片长度（字符数）
    chunk_overlap=50,                       # 重叠窗口
    separators=[                            # 分割符优先级（从高到低）
        "\n\n",                             # 段落边界
        "\n",                               # 换行
        "。", "！", "？",                    # 中文句末标点
        "；",                               # 分句
        "，",                               # 分句（降级最终边界）
    ],
    length_function=len,                    # 使用字符计数
    is_separator_regex=False,
)
with open('./百度千帆.txt','r',encoding='UTF-8') as f:
    text = f.read()
chunks = splitter.split_text(text)

# 统计每个切片的字符数
chunk_lengths = [len(chunk) for chunk in chunks]

# 基本统计量
print(f"切片数量: {len(chunks)}")
print(f"最小长度: {min(chunk_lengths)} 字符")
print(f"最大长度: {max(chunk_lengths)} 字符")
print(f"平均长度: {np.mean(chunk_lengths):.1f} 字符")
print(f"第一个切片内容：\n{chunks[0]}")
```

1.4.3 文档结构切片：概述

对于具有明确层次结构的文档（如Markdown、HTML、带标题的Word文档），利用文档结构进行切片可以最大限度地保留语义的完整性和层次关系。

解析文档的标题层级（H1、H2、H3等），以标题为锚点，将每个标题及其下属内容作为一个独立的语义单元。如果某个标题下的内容过长，再在该标题内部进一步细分。

（1）文档特征

技术文档通常具有清晰的层级结构，以 Markdown 格式存储，包含多级标题（H1~H6）、代码块、列表、表格等元素。此类文档的特点是：章节划分明确、主题聚焦、代码示例与说明文本交替出现。用户查询往往针对具体功能点、API参数或代码示例。本节我们使用“家用电视的使用说明”。

家用电视技术说明文档

本文档旨在详细说明家用电视的核心基础信息、使用方法、常见操作及基础维护要点，为用户提供全面、易懂的使用参考，确保用户能安全、便捷地操作电视，充分发挥电视的娱乐、资讯功能。家用电视作为家庭核心娱乐终端，已成为现代家庭不可或缺的设备，涵盖智能电视、传统液晶电视等主流类型，具备画面清晰、操作便捷、功能丰富等特点，广泛应用于家庭观影、休闲娱乐、资讯获取等场景。

一、产品概述

1.1 产品定义

家用电视是指用于家庭场景，以屏幕为显示载体，通过接收电视信号、网络信号或外接设备信号，实现图像、声音输出的视听设备，主要分为智能电视（支持网络连接、APP安装）和传统液晶电视（以接收有线/卫星信号为主），尺寸覆盖32英寸至85英寸，适配不同家庭空间大小。

1.2 核心用途与适用场景

核心用途：满足家庭成员观影、追剧、观看新闻资讯、播放外接设备内容（如U盘、游戏机）、亲子互动等需求，部分智能电视可实现视频通话、智能家居控制等拓展功能。

适用场景：客厅、卧室、书房等家庭室内空间，摆放需远离潮湿、高温、强光直射环境，避免影响设备寿命和观看体验，适配家庭日常休闲、节日团聚等各类场景。

（2）案例背景

某家电企业计划将《家用电视技术说明文档》Markdown 文档接入智能客服 RAG 系统，用于回答用户关于电视使用、参数、故障排查等问题。该文档约 5000 字，包含产品概述、技术参数、使用方法（开机/关机、遥控器操作、网络连接、外接设备等）、日常维护及常见故障排查等章节，结构清晰，以H2和H3标题划分主要内容。典型用户查询包括：“如何连接Wi-Fi？”“电视无法开机怎么办？”“HDMI接口在哪里？”等。

业务要求：检索系统需按功能点精准定位，避免跨章节信息混淆；切片长度需适配智能客服上下文（约 500 字符数），且不能截断代码块（如故障代码表）和表格（如参数列表）。

（3）切片策略选择

采用**文档结构切片为主、递归切片为辅**的组合策略。

- **主策略**：基于 Markdown 标题层级进行切片。以H2或H3标题作为语义单元边界，将每个标题及其下属内容作为一个独立Chunk。这样可保证每个切片聚焦于单一功能点或技术概念，便于精准检索。
- **辅助策略**：对于较长的章节（超过 500 字符数），在该章节内部使用递归切片进一步细分，优先以段落边界分割，必要时降级到句子边界。

（4） 关键参数配置

参数	配置值	说明
标题层级锚点	H2、H3	以二级或三级标题为切片单元，H1通常为文档总标题，过粗
最大切片长度	500 字符数	按照推荐最优区间进行取值
重叠窗口	50 字符数	保留标题与正文的衔接部分
分割符优先级	`["\n\n", "\n", "。", "；", " ", ""]`	优先段落边界，再降级到句子

1.4.4 文档结构切片：实现

LangChain 提供了 `MarkdownHeaderTextSplitter`，专门用于按 Markdown 标题层级进行结构化切片。

（1）基本用法

```python
from langchain_text_splitters import MarkdownHeaderTextSplitter

# 定义要切分的标题层级
headers_to_split_on = [
    ("#", "一级标题"),
    ("##", "二级标题"),
    ("###", "三级标题"),
]

# 创建切片器
markdown_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split_on,
    strip_headers=False  # 是否保留标题在内容中
)

# 执行切片
markdown_document = """
# Foo
## Bar
Hi this is Jim
Hi this is Joe
## Baz
Hi this is Molly
"""

splits = markdown_splitter.split_text(markdown_document)

for split in splits:
    print(f"元数据: {split.metadata}")
    print(f"内容: {split.page_content}\n")
```

输出效果

```
元数据: {'一级标题': 'Foo', '二级标题': 'Bar'}
内容: # Foo
## Bar
Hi this is Jim
Hi this is Joe

元数据: {'一级标题': 'Foo', '二级标题': 'Baz'}
内容: ## Baz
Hi this is Molly
```

（2）完整代码

```python
from langchain_text_splitters import (
    MarkdownHeaderTextSplitter,
    RecursiveCharacterTextSplitter
)
import numpy as np

def split_markdown_document(md_text: str, chunk_size: int = 512, overlap: int = 50):
    """技术文档专用切片器"""
    # 1. 按 H2/H3 标题切分
    headers_to_split_on = [
        ("##", "章节"),
        ("###", "小节"),
    ]
    md_splitter = MarkdownHeaderTextSplitter(headers_to_split_on, strip_headers=False)
    header_splits = md_splitter.split_text(md_text)

    # 2. 递归切分过长章节
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=overlap,
        separators=["\n\n", "\n", "。", "；", " ", ""],
    )
    final_chunks = text_splitter.split_documents(header_splits)

    return final_chunks
with open('家用电视技术说明文档.md', 'r', encoding='utf-8') as f:
    docs = f.read()

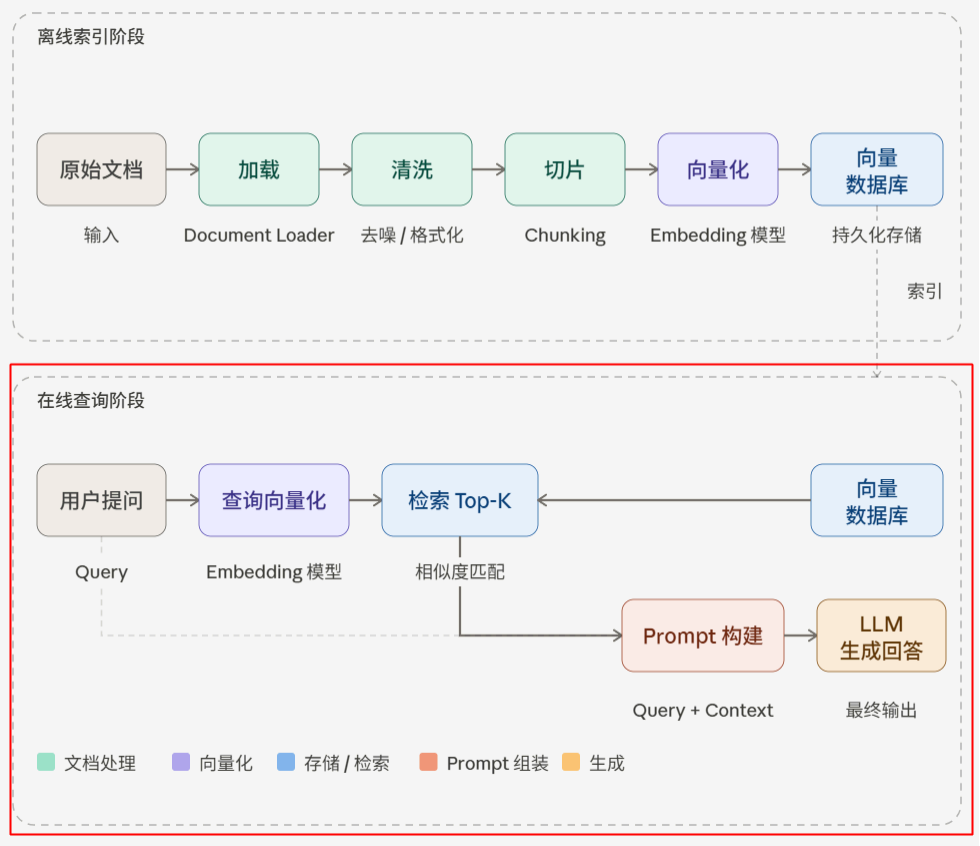

chunks = split_markdown_document(docs)

# 统计每个切片的字符数
chunk_lengths = [len(chunk.page_content) for chunk in chunks]

# 基本统计量
print(f"切片数量: {len(chunks)}")
print(f"最小长度: {min(chunk_lengths)} 字符")
print(f"最大长度: {max(chunk_lengths)} 字符")
print(f"平均长度: {np.mean(chunk_lengths):.1f} 字符")
print(f"第一个切片内容：\n{chunks[0].page_content}")
```

2. 在线查询阶段

离线索引阶段完成后，系统已经构建好可供检索的知识库。当用户提出问题时，系统需要实时完成检索与生成，这个过程称为**在线查询阶段**。



2.1.1 流程详解

1. 用户提问（User Query）

用户以自然语言的形式输入问题。这是整个在线查询阶段的起点。系统需要接收并理解用户的问题内容，为后续的向量化与检索做准备。在实际应用中，这一阶段可能还会进行简单的问题预处理，如去除多余空格或进行意图初步判断。

2. 查询向量化（Query Embedding）

将用户的问题通过与离线索引阶段相同的 Embedding 模型，转换为向量表示。这一过程非常关键，因为只有查询向量和文档向量处于同一语义空间中，才能进行有效的相似度检索。

3. 检索（Retrieval）

使用查询向量在向量数据库中进行相似度搜索，召回与问题最相关的 Top-K 个文档片段（Chunks）。检索是 RAG 系统的核心环节，其质量直接影响最终回答的准确性。此阶段可以采用纯向量检索、关键词检索或混合检索等方式。

4. Prompt构建（Prompt Construction）

将用户原始问题与检索到的文档片段进行组织，构建成完整的 Prompt 发送给大模型。通常需要将检索结果作为上下文（Context）插入，并配合系统提示词（System Prompt），明确告诉模型如何基于提供的知识进行回答，同时可加入引用格式要求等指令。

5. LLM生成回答（LLM Generation）

将构建好的 Prompt 输入大模型，由模型基于检索到的上下文信息生成最终回答。这一阶段模型不再仅依赖自身参数记忆，而是结合外部知识库进行生成，从而显著降低幻觉问题。

6.流程总结

在线查询阶段的完整流程如下：

```
用户提问 → 查询向量化 → 检索 → Prompt构建 → LLM生成回答
```

通过以上五个步骤，RAG 系统实现了“检索增强生成”的核心目标：让大模型在回答问题时能够参考外部知识库中的最新、准确信息，从而输出更可靠的结果。

2.2 针对用户提问：查询改写

在 RAG 的在线查询阶段，用户输入的问题往往比较口语化、简短或存在歧义，直接使用原始问题进行检索时，容易出现召回不全或相关性不足的情况。因此，在查询向量化之前，对用户问题进行改写是一种重要的优化手段。

2.2.1 概述

（1）什么是查询改写

查询改写（Query Rewriting）是指对用户原始问题进行改写、扩展或分解，生成一个或多个更适合向量检索的新查询版本。通过改写，可以使查询在语义上更加清晰、完整或覆盖更多相关维度，从而提升后续检索的效果。

查询改写通常由大模型完成，属于在线查询阶段的一种增强技术。

（2）为什么进行查询改写

用户原始问题在实际使用中往往存在以下问题：

- 问题过于简短，语义信息不足（如“苹果怎么样？”）
- 口语化表达或包含指代（如“它最近表现如何？”）
- 问题过于宽泛，难以精准匹配相关文档
- 存在潜在的多意图或多角度需求

直接使用这类问题进行检索，容易导致召回结果不准确或覆盖不全面。查询改写可以帮助系统更好地理解用户真实意图，从而提高检索的召回率和相关性。

2.2.2 单查询改写

单查询改写是指将用户原始问题改写成**一个**优化后的查询版本。

主要作用：

- 使问题更加清晰、完整和正式
- 补充必要的上下文或明确指代
- 将口语化问题转化为更适合检索的表达

常见改写方式：

- 补充上下文（如添加时间、地点、主体等信息）
- 明确指代对象
- 将问题转化为更结构化或更具体的表述
- 进行意图澄清

示例：

```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. 定义查询改写提示词模板
rewrite_prompt = ChatPromptTemplate.from_messages([
    ("system", """你是一个专业的查询改写助手。你的任务是将用户的原始问题改写成更适合向量检索的版本。
要求：
- 使问题更加清晰、完整、具体
- 补充必要的上下文或明确指代
- 去除口语化表达，使其更正式
- 只输出改写后的查询，不要输出任何解释或额外内容"""),
    ("human", "原始问题：{query}\n\n请改写为更适合检索的问题：")
])

# 2. 初始化 LLM（此处使用 DeepSeek）
llm = ChatOpenAI(
    model="deepseek-chat",
    temperature=0.3,            # 改写时建议使用较低温度，保持稳定性
    max_tokens=256
)

# 3. 构建查询改写链（LCEL 风格）
query_rewriter = rewrite_prompt | llm | StrOutputParser()

# 4. 执行查询改写
original_query = "苹果手机怎么样？"

# 调用改写链
rewritten_query = query_rewriter.invoke({"query": original_query})

print("原始问题：", original_query)
print("改写后的问题：", rewritten_query)
```

2.2.3 多查询改写

多查询改写（Multi-Query Rewriting）是指将用户的一个问题改写成**多个**不同角度或不同表述的子查询，然后分别进行检索，最后对结果进行融合。

主要作用：

- 从多个维度覆盖用户潜在的信息需求
- 提高检索的召回率，减少单一查询可能遗漏的重要信息
- 特别适合复杂、开放性或多意图的问题

常见做法：

- 使用大模型生成多个不同表述的查询
- 每个查询独立进行向量检索
- 对多路检索结果进行去重、融合或重排序

示例:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser

# 初始化解析器
parser = JsonOutputParser()

# 构建提示词（直接把 JSON 格式写死）
multi_query_prompt = ChatPromptTemplate.from_messages([
    ("system",
     """你是一个专业的查询改写助手。
请将用户的问题改写成 **3个不同角度** 的检索查询。

请严格按照以下 JSON 格式输出，不要输出任何解释：

{
  "queries": [
    "查询1",
    "查询2",
    "查询3"
  ]
}"""),
    ("human", "原始问题：{query}")
])

llm = ChatOpenAI(
    model="deepseek-chat",
    temperature=0.7,
    max_tokens=512
)

# 构建链
multi_query_chain = multi_query_prompt | llm | parser

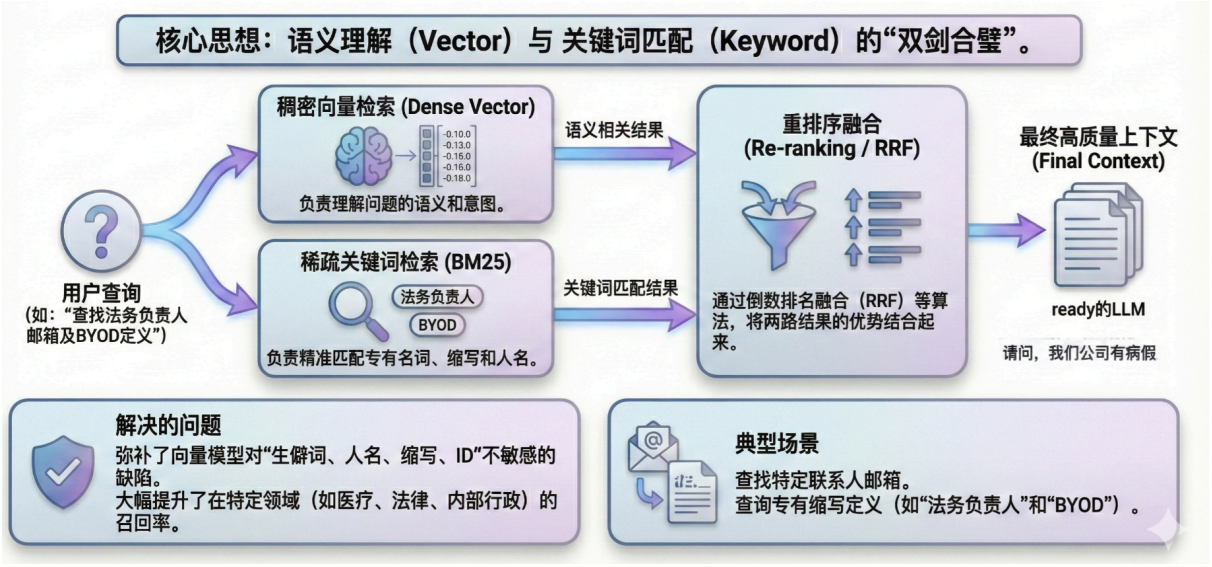
original_query = "苹果手机怎么样？"

result = multi_query_chain.invoke({"query": original_query})

print("原始问题：", original_query)
print("改写后的查询列表：", result["queries"])
```

2.3 混合检索

在 RAG 系统中，检索环节是决定最终回答质量的关键步骤。不同的检索方式各有优势和局限，单一检索方式往往难以兼顾语义理解和精确匹配。因此，混合检索成为目前主流且有效的检索策略。



2.3.1 常见的检索方式

（1）精确匹配（Exact Match）

精确匹配是指通过字符串完全一致或规则匹配的方式进行检索。这种方式对查询和文档中的词语要求严格匹配，适合需要高度精确的结果场景，但召回率较低，对语义理解能力较弱。

（2）关键字检索（Keyword Retrieval）

关键字检索通常基于词频和倒排索引实现，经典算法包括 BM25 等。它能够有效捕捉关键词的出现频率和位置信息，在处理精确术语、专业词汇时表现较好。但其局限在于无法理解语义相似性，对同义词、近义词的处理能力较弱。

（3）纯向量检索（Dense Vector Retrieval）

纯向量检索通过 Embedding 模型将文本转化为稠密向量，再通过向量相似度（如余弦相似度）进行检索。这种方式擅长捕捉语义层面的相似性，即使查询和文档使用不同的词汇表达相同含义，也能召回相关内容。但在处理精确匹配、稀疏关键词时效果往往不如关键字检索。

2.3.2 混合检索

（1）什么叫做混合检索

混合检索（Hybrid Retrieval）是指同时使用多种检索方式（通常是**关键字检索**和**向量检索**）进行召回，并将两路结果进行融合排序，最终返回综合排序后的文档片段。

它不是简单地把两种检索结果拼接，而是通过特定的融合策略，对不同来源的检索结果进行重新排序和筛选。

（2）为什么使用混合检索

单一检索方式存在明显短板：

- 纯向量检索虽然语义能力强，但容易忽略精确关键词，导致某些关键信息召回失败。
- 关键字检索虽然精确度高，但缺乏语义理解能力，难以处理同义表达和复杂语义。

混合检索结合了两者的优势，能够在**语义相似性**和**关键词精确性**之间取得更好的平衡，从而提升整体检索质量。

（3）混合检索的架构

典型的混合检索架构如下：

1. 用户问题同时送入**关键字检索器**和**向量检索器**进行并行检索。
2. 两路检索器各自返回 Top-K 结果。
3. 通过融合策略（RRF或WLC）对两路结果进行打分合并与重排序。
4. 返回最终融合后的 Top-K 文档片段用于后续生成。

2.3.3 关键词检索：BM25算法

BM25 是目前关键词检索（稀疏检索）中最主流的排序算法，它是在 **TF-IDF** 的基础上改进而来的。下面我们从直观理解到公式，逐步讲解清楚。

（1）BM25 的核心思想

BM25 的打分逻辑可以总结为以下三点：

- **词频（TF）**：一个词在文档中出现的次数越多，这个词对文档的重要性越高。
- **逆文档频率（IDF）**：一个词在整个文档集合中出现的文档越少（越稀有），它就越重要。
- **文档长度惩罚**：如果文档很长，即使某个词出现了很多次，也不能说明这个词一定很重要（因为长文档天然更容易出现某个词）。因此需要对长文档进行惩罚。

简单来说，**BM25 认为一个好的匹配应该是"又相关又简洁"的文档。**

（2）BM25 的简化公式

BM25 对每个查询词（Query Term）分别计算得分，最后将所有词的得分相加，得到文档的最终 BM25 分数。

单个词的 BM25 得分公式如下（简化版）：

$$\mathrm{BM25}(d, q_i) = \frac{\mathrm{TF}(q_i, d) \cdot (k_1 + 1)}{\mathrm{TF}(q_i, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}})} \cdot \mathrm{IDF}(q_i)$$

其中各参数含义如下：

参数	含义	作用
TF	词 q_i 在文档 d 中出现的次数	词频越高，分数越高
IDF	词 q_i 的逆文档频率	词越稀有，分数越高
|d|	当前文档的长度（词数）	文档越长，需要更高词频
avgdl	所有文档的平均长度	用于长度归一化
k_1	通常取 1.2~2.0（经验值）	控制词频饱和度
b	通常取 0.75	控制文档长度惩罚强度

（3）BM25 vs TF-IDF

对比项	TF-IDF	BM25	优势
词频处理	线性增长	有饱和机制	BM25 更合理
文档长度	不考虑	有长度惩罚	BM25 更公平
排序效果	一般	显著更好	BM25 更优
计算复杂度	低	稍高	TF-IDF 更快
实际使用	多用于特征提取	现代检索系统主流排序算法	BM25 更常用

（4）与向量检索得分的对比

在混合检索中，向量检索和关键词检索（BM25）是两种完全不同的打分机制，它们的得分特性存在明显差异，这直接导致了融合策略的设计需要特别考虑。

检索方式	得分范围	是否有上限	原因	在混合检索中的影响
向量检索	通常 0~1	有	使用余弦相似度归一化	得分范围统一，易于融合
BM25	0 ~ 几十（无固定上限）	无	基于词频和IDF累加	得分范围差异大，必须归一化后才能融合

- **向量检索**使用的是**余弦相似度**，其得分被天然限制在 [0, 1] 区间，具有良好的可比性。
- **BM25**是多个查询词得分的**累加结果**，且没有进行全局归一化。语料库大小、查询词数量、文档长度等因素都会影响其绝对数值，导致不同场景下的 BM25 得分不具备直接可比性。

（5）融合时的核心挑战

由于两种检索方式的得分范围和计算逻辑完全不同，如果直接将两者得分相加或比较，权重再怎么调整都难以公平体现两者的贡献。因此，在进行混合检索时，**必须采用特殊的融合策略**来解决得分不一致的问题。

目前主流的两种融合策略如下：

- RRF 核心策略：完全不依赖具体得分，只看文档在各检索通道中的排名位置。
- WLC 核心策略：对不同检索通道的得分进行归一化后，再按预设权重进行加权求和。

2.3.4 融合策略：互惠秩融合（Reciprocal Rank Fusion, RRF）

RRF是目前RAG系统和混合检索中最受欢迎、应用最广泛的融合方法，尤其适合向量检索和关键词检索这种异构检索器的组合。

（1）**核心思想**：

RRF**完全不依赖具体的得分数值**，而是仅根据文档在每一路检索结果中的**排名位置（Rank）**来计算融合得分。排名越靠前，赋予的权重越高，从而实现“位置越前越重要”的直观逻辑。

（2）**计算公式**：

$$\mathrm{RRFScore}(d) = \sum_{i=1}^{m} \frac{1}{k + r_i(d)}$$

其中：

- m 为检索路径数量（此处m=2，即向量检索和关键词检索）
- $r_i(d)$ 表示文档 d 在第 i 路检索结果中的排名位置（第1名记为1，第2名记为2，以此类推）
- k 是平滑常数，通常固定取60（大量实验验证的最优经验值，用于防止排名靠后的文档得分过低，同时降低对Top位置的过度敏感）

（3）RRF 计算示例

假设我们有 **5 个文档片段**（Chunk），分别为：Chunk_A ~ Chunk_E

现在分别进行**向量检索**和**关键词检索**，各自返回 Top 5 的结果及排名如下：

排名	向量检索结果	关键词检索结果
1	Chunk_C	Chunk_A
2	Chunk_A	Chunk_D
3	Chunk_B	Chunk_B
4	Chunk_E	Chunk_C
5	Chunk_D	Chunk_E

（4）RRF 计算过程

下面我们分别计算每个 Chunk 的 RRF 得分（取 `k=60`）：

Chunk	向量检索排名	关键词检索排名	RRF 计算公式	RRF 得分	说明
Chunk_A	2	1	1/(2+60) + 1/(1+60)	**0.0323**	在两个检索中都排名靠前，得分最高
Chunk_B	3	3	1/(3+60) + 1/(3+60)	**0.0317**	两个检索排名相同，综合得分较高
Chunk_C	1	4	1/(1+60) + 1/(4+60)	**0.0320**	向量检索排名第1，综合得分也很高
Chunk_D	5	2	1/(5+60) + 1/(2+60)	**0.0310**	关键词检索排名靠前，依然获得不错得分

Chunk	向量检索排名	关键词检索排名	RRF 计算公式	RRF 得分	说明
Chunk_E	4	5	1/(4+60) + 1/(5+60)	**0.0305**	两个检索排名都较后，得分相对较低

（5）简化理解

RRF 的核心思想可以简单理解为：

一个文档在多个检索通道中的排名越靠前，它的综合得分就越高。即使某个文档只在其中一个检索通道中出现，它依然能获得一定的分数，而不会被直接淘汰。

这种机制既奖励**多通道都表现优秀**的文档，也给**单一通道表现突出**的文档保留机会。

2.3.5 加权线性融合（Weighted Linear Combination）

线性加权融合是混合检索中另一种常用的结果融合方法。它通过为不同检索通道分配不同的权重，并将各通道的得分进行加权求和，从而得到最终的综合得分。

（1）核心思想

线性加权融合的核心思想是：**不同检索方式的重要性是不同的**，可以通过人为设定权重来控制它们对最终排序的影响。权重越高的检索方式，其结果对最终排序的影响越大。

与 RRF 仅依赖排名不同，线性加权融合直接使用各检索器的原始得分（或归一化后的得分），因此能够更精细地反映不同检索通道的置信度差异。

（2）计算公式

线性加权融合的计算公式如下：

$$\text{Score}(d) = w_1 \cdot s_1(d) + w_2 \cdot s_2(d) + \cdots + w_m \cdot s_m(d)$$

其中：

- m 为检索通道数量（此处 $m = 2$，即向量检索和关键词检索）
- w_i 表示第 i 路检索的权重（所有权重之和通常为 1）
- $s_i(d)$ 表示文档 d 在第 i 路检索中的得分（一般需要进行归一化处理，使不同检索器的得分具有可比性）

（3）线性加权融合计算示例

我们继续使用之前的 5 个文档片段（Chunk_A ~ Chunk_E），假设向量检索和关键词检索返回的**MinMax-Score得分**（范围 0~1）如下：

Chunk	向量检索得分	关键词检索得分
Chunk_A	0.92	0.85
Chunk_B	0.78	0.71
Chunk_C	0.95	0.62

Chunk	向量检索得分	关键词检索得分
Chunk_D	0.65	0.88
Chunk_E	0.71	0.55

（4）线性加权融合计算过程

假设我们设定以下权重：

- 向量检索权重 $w_1 = 0.6$
- 关键词检索权重 $w_2 = 0.4$

下面计算每个 Chunk 的综合得分：

Chunk	向量检索得分	关键词检索得分	计算公式	综合得分	说明
Chunk_A	0.92	0.85	$0.6 \times 0.92 + 0.4 \times 0.85$	**0.892**	两个通道得分都较高，综合得分最高
Chunk_B	0.78	0.71	$0.6 \times 0.78 + 0.4 \times 0.71$	**0.752**	得分较为均衡
Chunk_C	0.95	0.62	$0.6 \times 0.95 + 0.4 \times 0.62$	**0.818**	向量检索得分极高，综合得分较高
Chunk_D	0.65	0.88	$0.6 \times 0.65 + 0.4 \times 0.88$	**0.742**	关键词检索得分较高，拉高了综合得分
Chunk_E	0.71	0.55	$0.6 \times 0.71 + 0.4 \times 0.55$	**0.646**	两个通道得分均较低，综合得分最低

通过调整权重，我们可以灵活控制向量检索和关键词检索对最终排序的影响程度。

（5）简化理解

线性加权融合的核心思想可以理解为：

> **给不同的检索通道分配不同的“重要性权重”，然后把它们的得分按权重相加。**

这种方法灵活性强，可以根据业务场景调整向量检索和关键词检索的占比。但缺点是需要人工调参，且不同检索器的得分需要先进行归一化，否则权重设置会失去意义。

相比 RRF，线性加权融合更依赖得分质量，而 RRF 更依赖排名位置，两者各有适用场景。